Diseño de Software de la Plataforma EduAirControl

María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas

Tecnólogo en Análisis y Desarrollo de Software – 3145556, Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software – 3145556

Todos los instructores del programa

15 de octubre de 2026 · 2025-2026

Resumen

Este documento presenta el diseño de software de EduAirControl, una plataforma de monitoreo y análisis ambiental para ambientes educativos construida como un sistema de microservicios. El diseño se organiza en cuatro vistas: interfaces gráficas, modelo arquitectónico, patrones de diseño y modelo de datos. La vista de interfaces documenta el sistema de diseño (tokens de color, tipografía, espaciado y componentes), el inventario de doce pantallas y las capturas de los mockups de escritorio y móvil, junto con los mapas de navegación y la matriz de acceso por rol. La vista arquitectónica describe el estilo de microservicios con API Gateway y comunicación orientada a eventos, el catálogo de servicios con sus puertos, los flujos de comunicación sincrónica (REST) y asincrónica (RabbitMQ con outbox e inbox), y la estrategia de seguridad basada en JWT RS256 y JWKS. La vista de patrones traza cada patrón adoptado — hexagonal de puertos y adaptadores, repositorio, API Gateway, BFF, outbox transaccional, inbox idempotente, CQRS de lectura, circuit breaker documentado— con su implementación concreta en el código. La vista de datos resume los seis esquemas PostgreSQL, sus tablas principales y las reglas de modelamiento. El diseño se elaboró sobre el código ejecutable y la documentación de gobierno del proyecto; cuando difieren, se documenta la desviación. El documento sirve como referencia para la implementación, las pruebas y la operación del sistema.

Palabras clave: diseño de software, arquitectura de microservicios, patrones de diseño, interfaces gráficas, modelo de datos, hexagonal.

Índice

Resumen 2

1. Introducción 4

2. Interfaces gráficas 5

    2.1 Sistema de diseño 5

    2.2 Inventario de pantallas 7

    2.3 Pantallas principales 9

    2.4 Navegación y control de acceso 19

3. Modelo arquitectónico 20

    3.1 Estilo arquitectónico adoptado 20

    3.2 Vista de contexto y de contenedores 20

    3.3 Catálogo de servicios 21

    3.4 Comunicación entre servicios 22

    3.5 Seguridad 23

    3.6 Arquitectura interna de los servicios 23

4. Patrones de diseño 24

    4.1 Patrones arquitectónicos y de microservicios 24

    4.2 Patrones GoF presentes en el código 25

    4.3 Trazabilidad de patrones en la estructura de paquetes 26

5. Modelo de datos 27

    5.1 Principios de modelamiento 27

    5.2 Esquemas y tablas principales 27

    5.3 Reglas de negocio del modelo 28

6. Diagramas de implementación 30

7. Conclusiones 30

Referencias 31

Diseño de Software de la Plataforma EduAirControl

1. Introducción

El presente documento constituye el diseño de software del proyecto EduAirControl, en el marco del programa de Análisis y Desarrollo de Software (ADSO). Su propósito es especificar cómo se construye el sistema: su lenguaje visual y sus pantallas, su estructura arquitectónica, los patrones que organizan el código y su modelo de datos. El diseño traduce las decisiones del análisis (Entregable 03) y de la propuesta técnica (Entregable 04) en una descripción verificable contra el código implementado.

El documento se elaboró tomando como fuente primaria el repositorio ejecutable del proyecto —`apps/web` y `apps/mobile` del frontend, los seis microservicios Spring Boot, el API Gateway y el stack de infraestructura— y como fuente secundaria la documentación de gobierno (`docs-ea-control`), los contratos de API y el script de base de datos (Entregable 06). Cuando la documentación y el código difieren, se describe lo que hace el código y la desviación se señala explícitamente.

El documento está dirigido al equipo de desarrollo, al instructor evaluador y a los equipos de pruebas y operación. Se organiza como sigue: Sección 2 (interfaces gráficas), Sección 3 (modelo arquitectónico), Sección 4 (patrones de diseño), Sección 5 (modelo de datos), Sección 6 (diagramas de implementación), Sección 7 (conclusiones) y referencias.

Tabla 1

Entregables relacionados con este documento

Entregable	Contenido	Uso en este documento
`01_Documento_General_Proyecto`	Visión, alcance y estado del proyecto	Contexto general
`02_Especificacion_Requisitos`	Requisitos `RFx.y`, `RNFx` y casos de uso	Trazabilidad de pantallas y servicios
`03_Analisis_Software`	Análisis, vistas dinámicas y clases	Insumo directo de este diseño

Entregable	Contenido	Uso en este documento
`04_Propuesta_Tecnica`	Decisiones tecnológicas	Justificación del modelo arquitectónico
`06_Base_Datos`	Modelo entidad-relación y script SQL	Sección 5 (modelo de datos)
`07_Pruebas` a `15_Enlace_Repositorio_GitHub`	Pruebas, calidad, manuales y anexos	Consumidores aguas abajo del diseño

2. Interfaces gráficas

La vista de interfaces documenta el lenguaje visual de EduAirControl y sus pantallas. La fuente de verdad es el código del frontend (`ea-control-frontend`), del que se extrajeron los tokens de diseño reales; los mockups de alta fidelidad replican esos tokens para validación de layout.

2.1 Sistema de diseño

El sistema de diseño define las variables visuales compartidas entre las aplicaciones web y móvil. Los tokens viven en CSS custom properties (`apps/web/src/index.css`) y se replican uno a uno en el móvil (`tokens.js`). El componente reutilizable principal es la familia `.ds-*` de `design-system.css`.

Tabla 2

Tokens de diseño principales

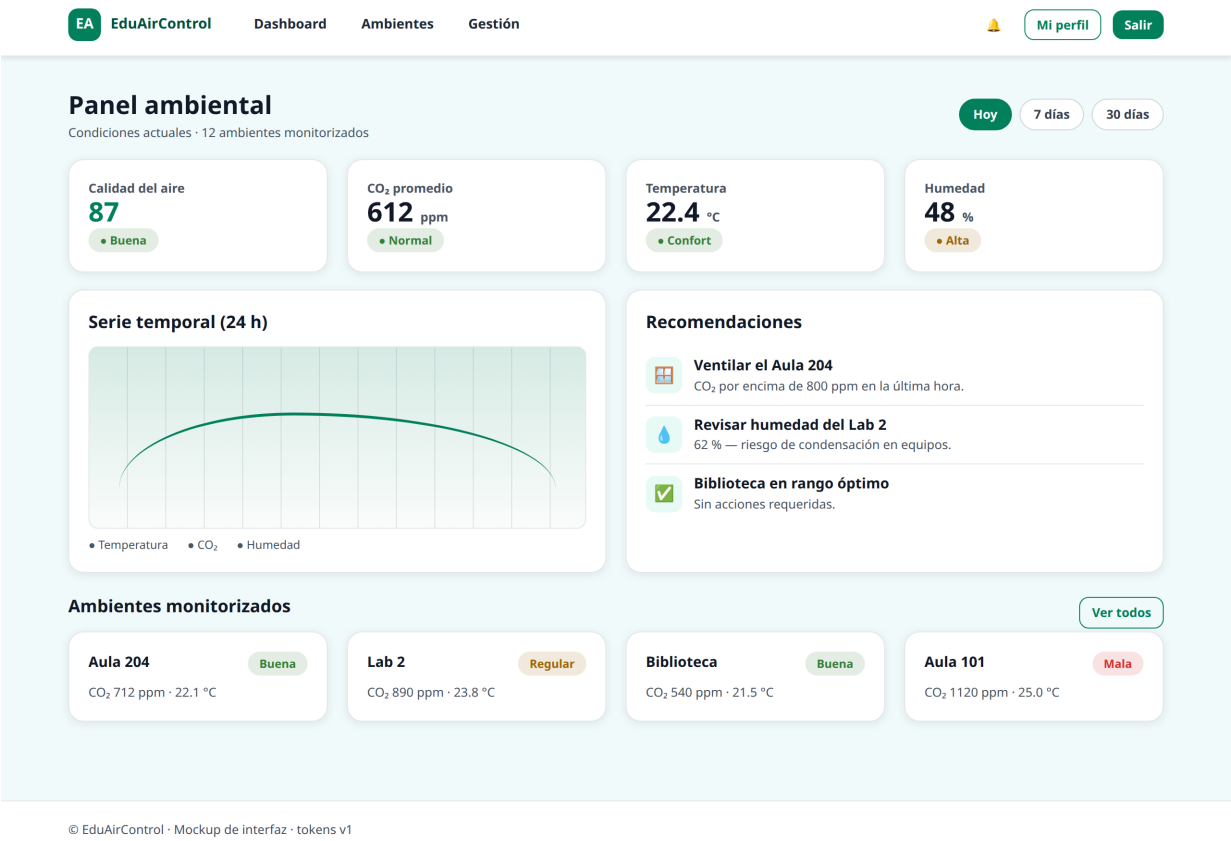
Categoría	Token	Valor (tema claro)	Uso
Acento	`--accent`	`#01805b`	Botones primarios, enlaces, navegación activa
Acento	`--accent-light`	`#e8faf5`	Superficies tintadas suaves
Semántico	`--color-success`	`#368139`	Confirmaciones, estado bueno
Semántico	`--color-warning`	`#9d6602`	Advertencias, estado regular

Categoría	Token	Valor (tema claro)	Uso
Semántico	`--color-danger`	`#d63030`	Errores, estado malo
Texto	`--text-primary`	`#111827`	Texto principal
Superficie	`--bg-body` / `--bg-card`	`#f0fafa` / `#ffffff`	Fondo de página / tarjetas
Espaciado	`--space-xs` … `--space-xl`	6 · 12 · 20 · 32 · 48 px	Ritmo de layout
Radio	`--radius-sm` … `--radius-full`	10 · 16 · 20 · 28 · 999 px	Bordes de tarjetas y píldoras
Capas	`--z-panel` … `--z-loading`	100 · 200 · 300 · 400 · 500	Orden de apilamiento

El modo oscuro es un tema de primera clase (`body.dark-mode`), no una inversión de colores: redefine fondos, textos, bordes y sombras. Además de los tres temas de accesibilidad cromática (protanopia `#007aa7`, deuteranopia `#4f7e00`, tritanopia `#5b4b8a`), el sistema contempla escalado de texto (`--a11y-font-size` de 16, 18 y 20 px) y respeta `prefers-color-scheme` cuando el usuario no ha elegido tema.

Figura 1

*Pantalla de dashboard ambiental (escritorio).*Captura del mockup de escritorio con los tokens del sistema de diseño: tarjetas de métricas, serie temporal, recomendaciones y lista de ambientes.**



2.2 Inventario de pantallas

El sistema consta de doce pantallas funcionales. Once están presentes en web y once en móvil; dos pantallas son específicas de plataforma: la página de aterrizaje es solo web y el provisionamiento de sensores es solo móvil.

Tabla 3

Inventario de pantallas y su navegación

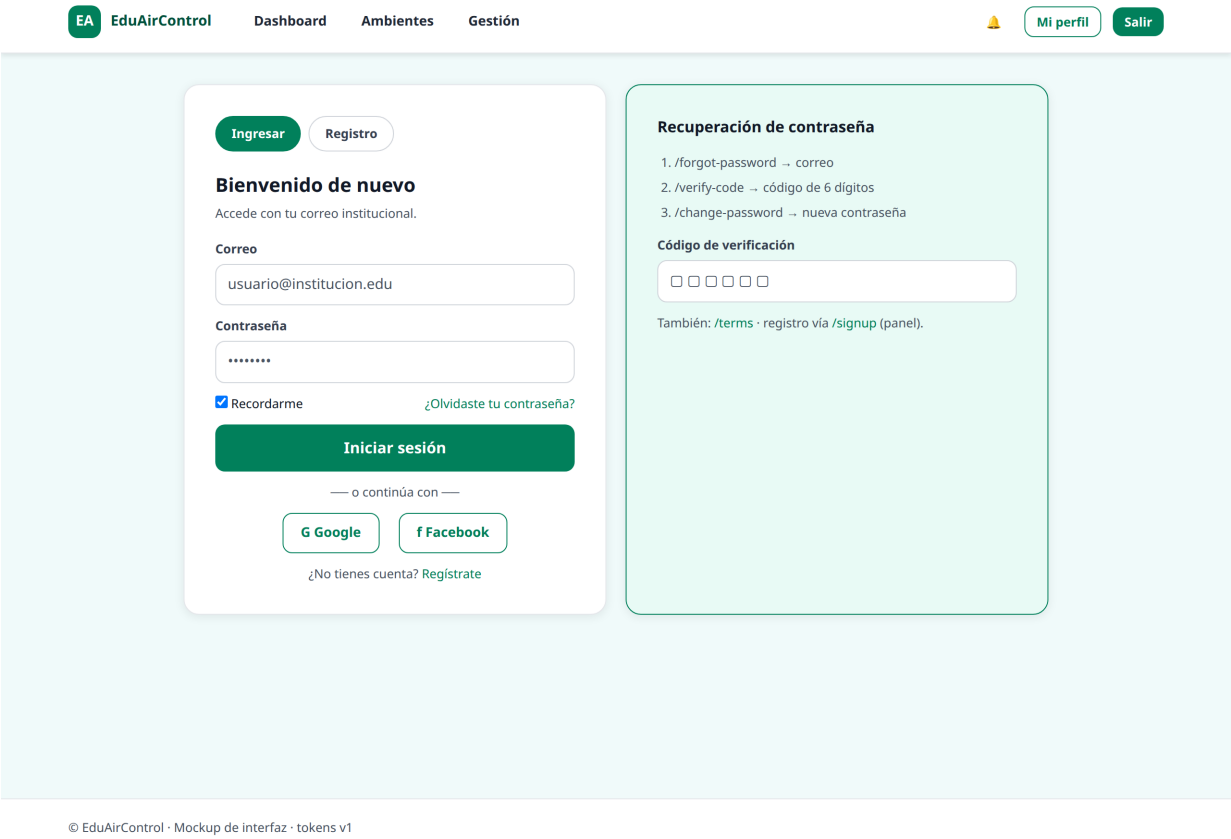
#	Pantalla	Web	Móvil	Ruta / navegación	Rol mínimo
01	Página de aterrizaje	✅	—	`/landing`	Público
02	Autenticación	✅	✅	`/login` (+ recuperación)	Invitado
03	Guía de producto	✅	✅	`/guide`	Público
04	Dashboard ambiental	✅	✅	`/dashboard`	USER
05	Ambientes educativos	✅	✅	`/all-environments`, `/environment/:id`	USER
06	Ambientes favoritos	✅	✅	`/favorites`	USER
07	Administración de ambientes	✅	✅	`/management`	ADMIN
08	Panel super administrador	✅	✅	`/admin/institutions`	SUPER_ADMIN
09	Notificaciones de alertas	panel lateral	pantalla	overlay / `NotificationsPanel`	USER
10	Perfil de usuario	✅	✅	`/profile`	USER
11	Configuración de usuario	✅	✅	`/settings`	USER
12	Provisionamiento de sensores (BLE)	—	✅	stack `Provisioning`	ADMIN

2.3 Pantallas principales

La autenticación concentra el ingreso, el registro y la recuperación de contraseña (cuatro pantallas encadenadas: `/forgot-password`, `/verify-code`, `/change-password`) e incluye inicio de sesión social con Google y Facebook mediante flujo de código de un solo uso.

Figura 2

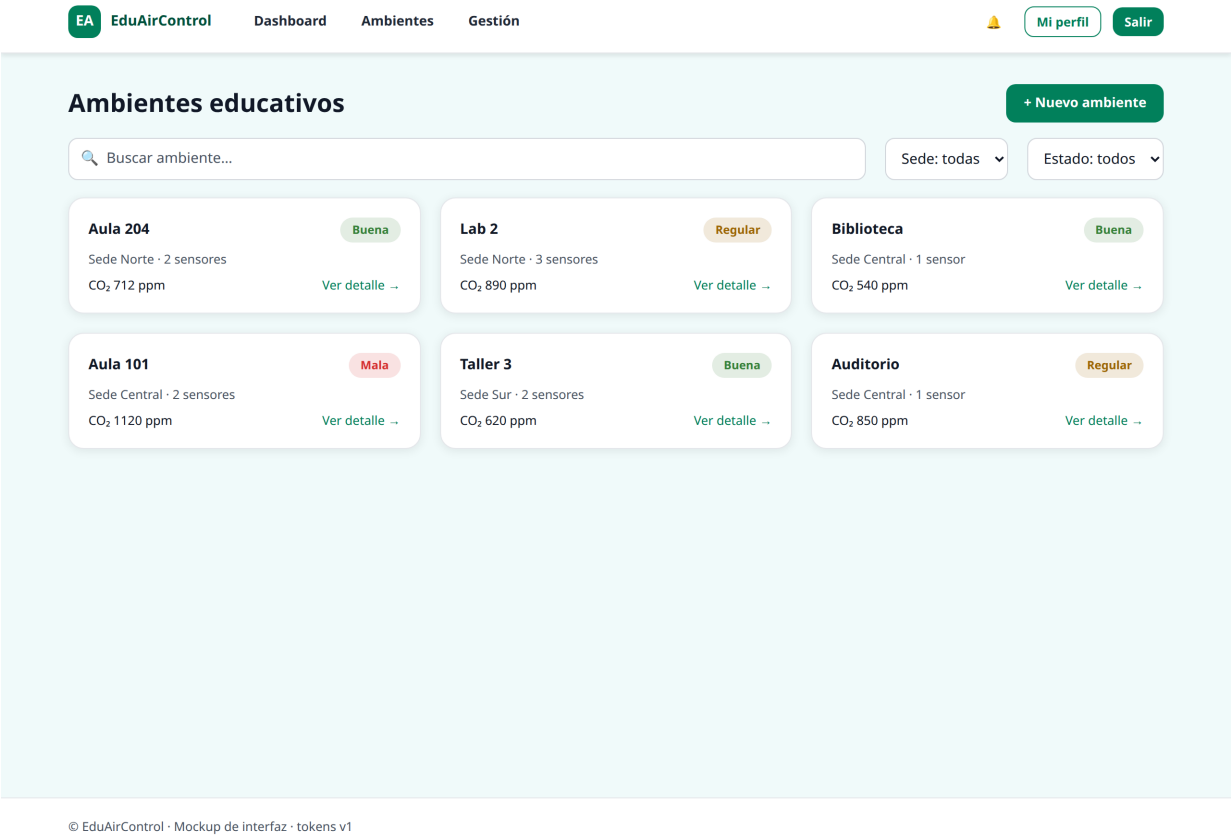
*Pantalla de autenticación (escritorio).*Paneles de ingreso y registro, inicio social y flujo de recuperación de contraseña.**



El listado de ambientes educativos reutiliza el componente `EnvironmentCard` en tres contextos distintos (dashboard, listado y favoritos) y aplica filtros por sede y estado.

Figura 3

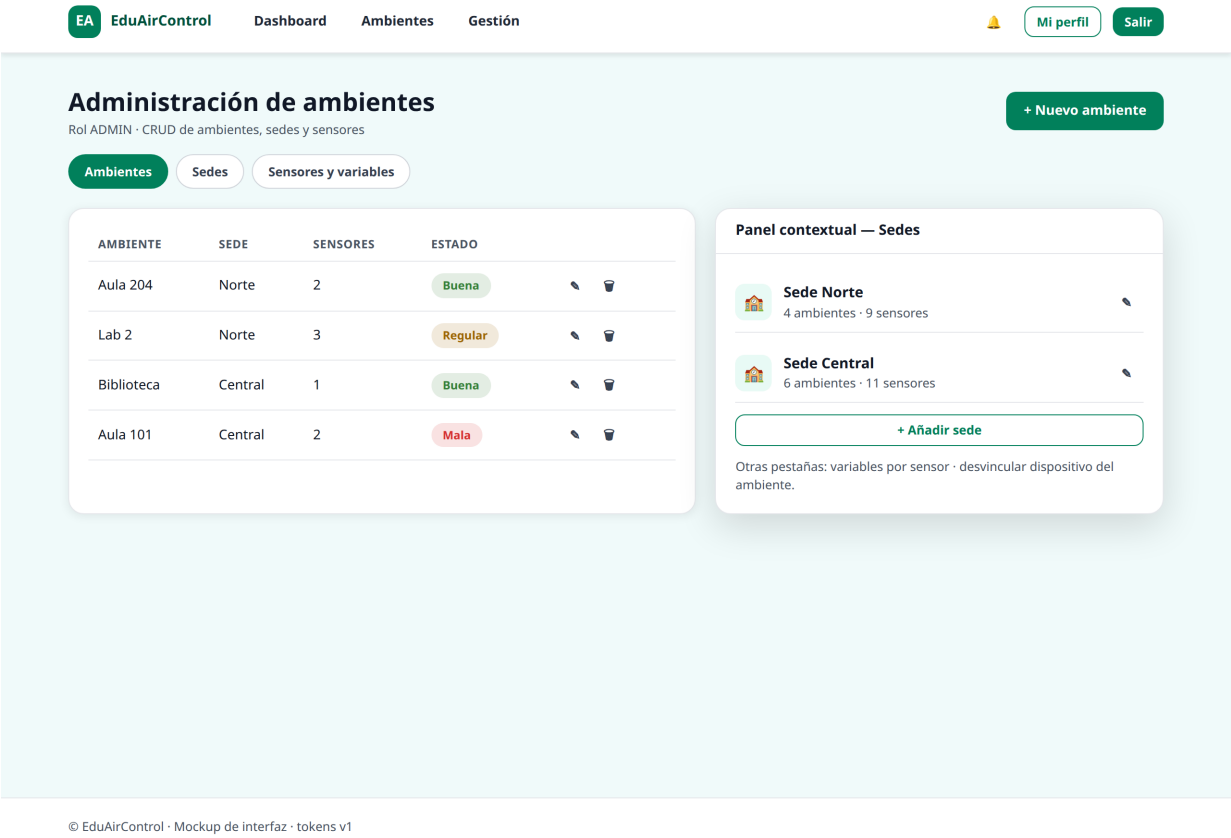
*Listado de ambientes educativos (escritorio).*Tarjetas reutilizables con estado de calidad, filtros de búsqueda y navegación al detalle.**



La administración de ambientes (rol ADMIN) concentra el CRUD de ambientes, la gestión de sedes, el panel de variables por sensor y la desvinculación de dispositivos. En móvil añade el provisionamiento BLE del ESP32.

Figura 4

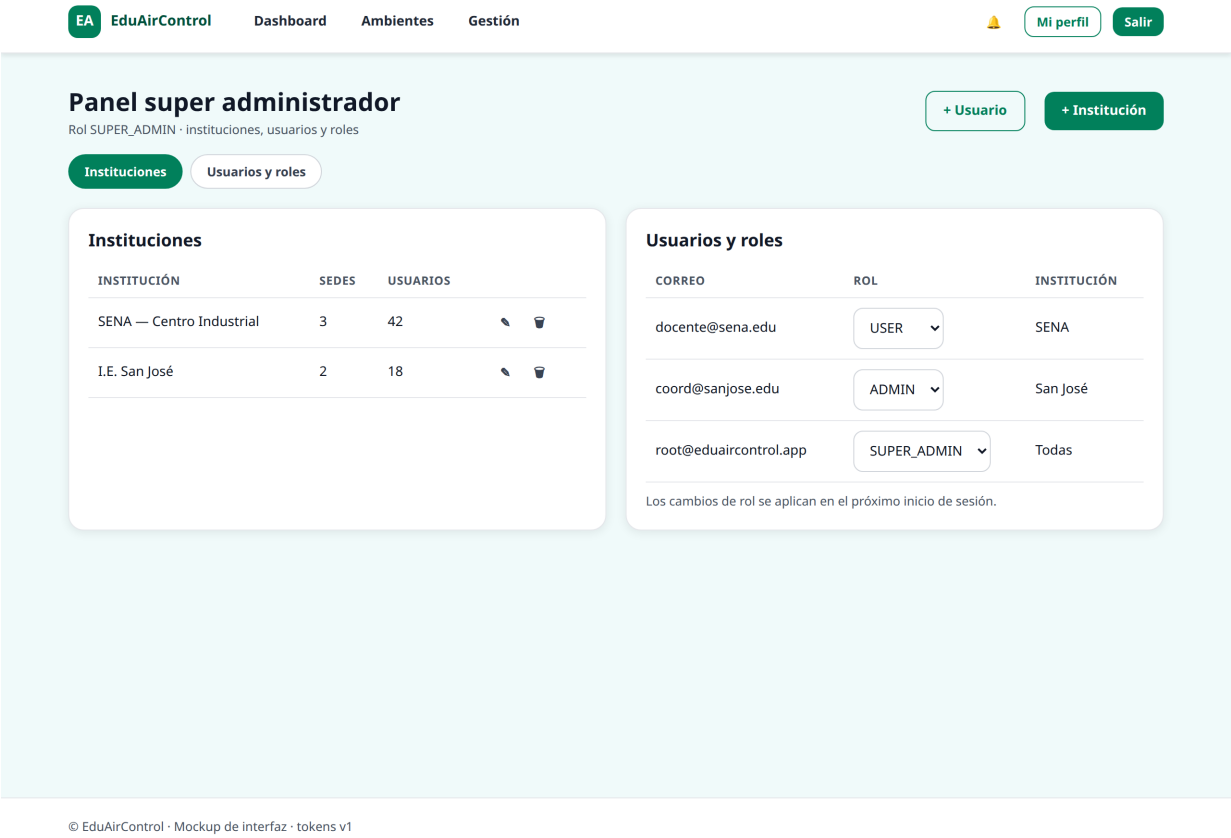
*Administración de ambientes (escritorio).*Tabla de ambientes con acciones de edición y borrado, y panel contextual de sedes.**



El panel de super administrador (rol SUPER_ADMIN) integra la gestión de instituciones y la asignación de roles de usuario en una sola pantalla.

Figura 5

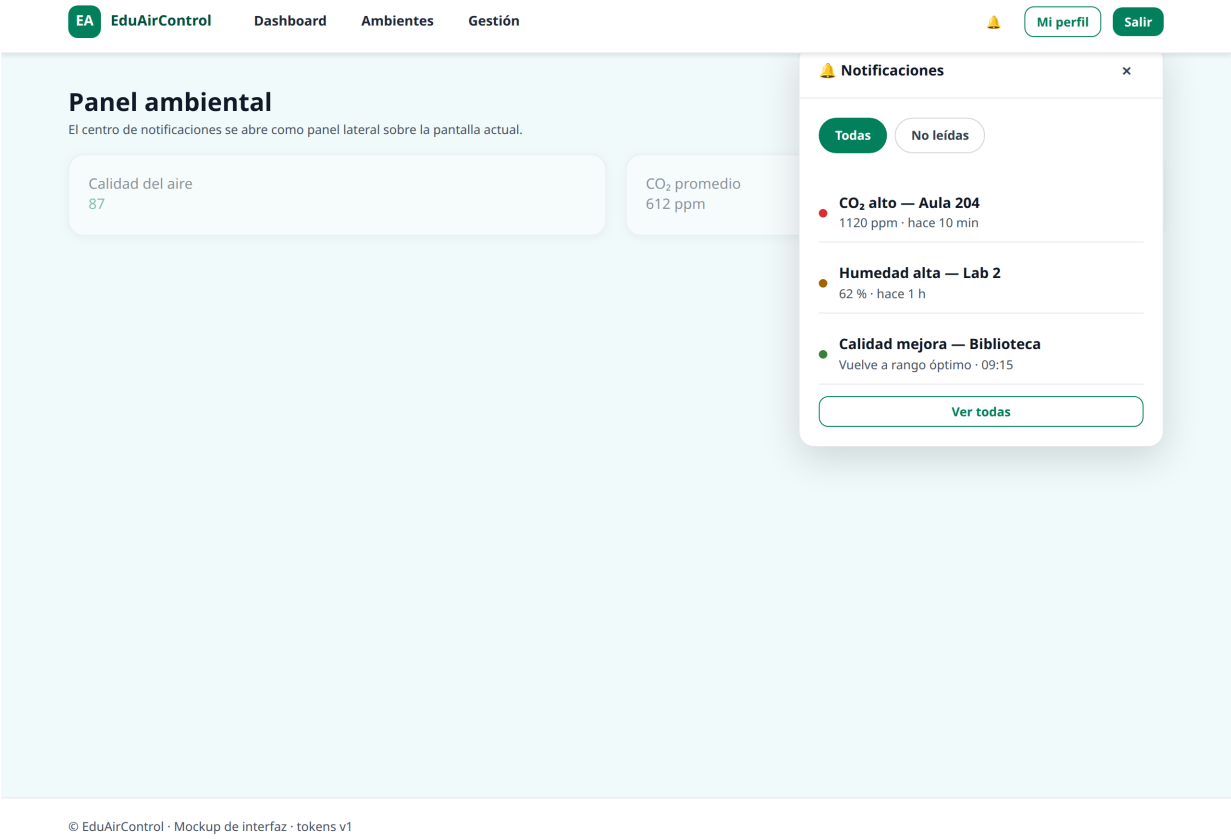
*Panel super administrador (escritorio).*Gestión de instituciones y de usuarios con asignación de roles USER, ADMIN y SUPER_ADMIN.**



Las notificaciones se presentan como panel lateral superpuesto en web y como pantalla en móvil: la misma funcionalidad con dos patrones de navegación distintos, decisión deliberada por la disponibilidad de espacio en cada plataforma.

Figura 6

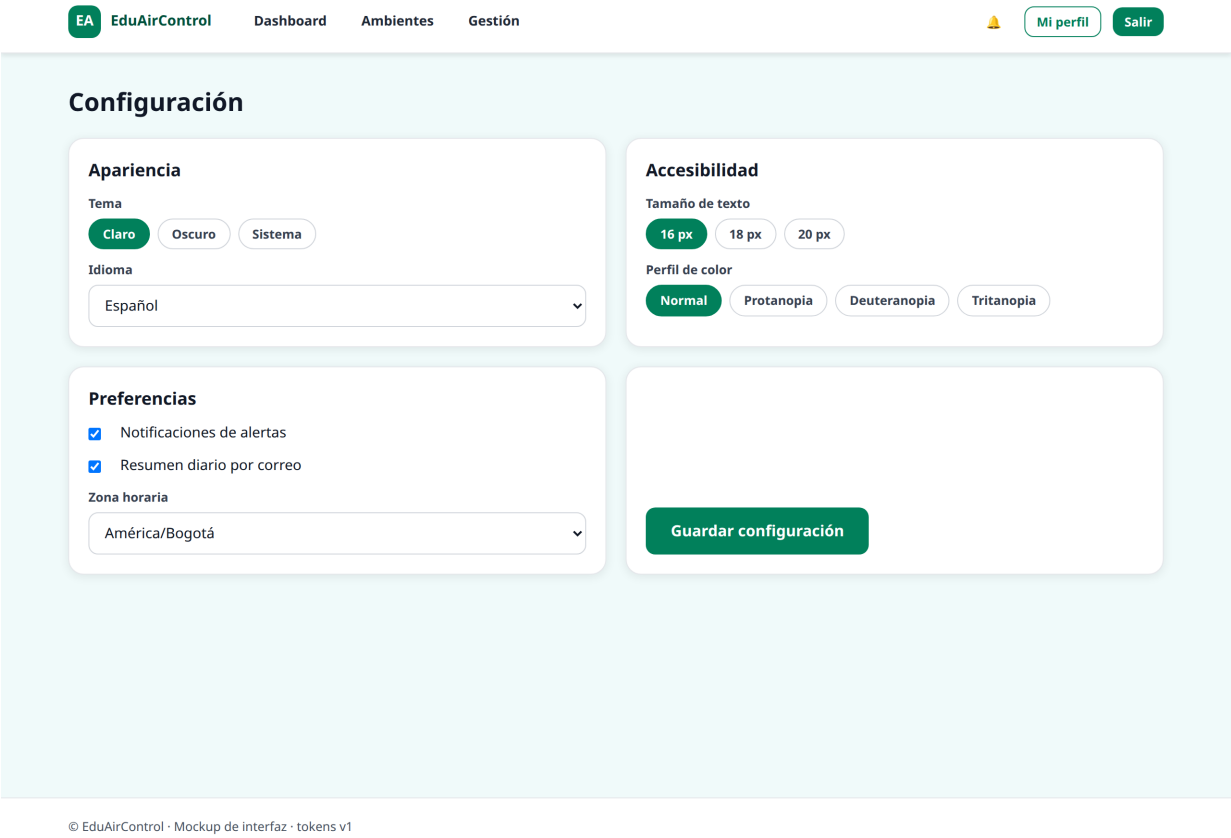
*Centro de notificaciones como panel lateral (escritorio).*Alertas por severidad con navegación al ambiente que las originó.**



La configuración agrupa apariencia (tema claro/oscuro/sistema), idioma (español, inglés, portugués, francés) y accesibilidad (tamaño de texto y perfil de color).

Figura 7

*Configuración de usuario (escritorio).*Tema, idioma y opciones de accesibilidad basadas en los tokens del sistema de diseño.**



En la aplicación móvil, el dashboard resume las métricas del ambiente y el provisionamiento BLE guía la vinculación del ESP32 en tres pasos: escaneo, configuración de red y asociación al ambiente.

Figura 8

Dashboard ambiental (móvil).

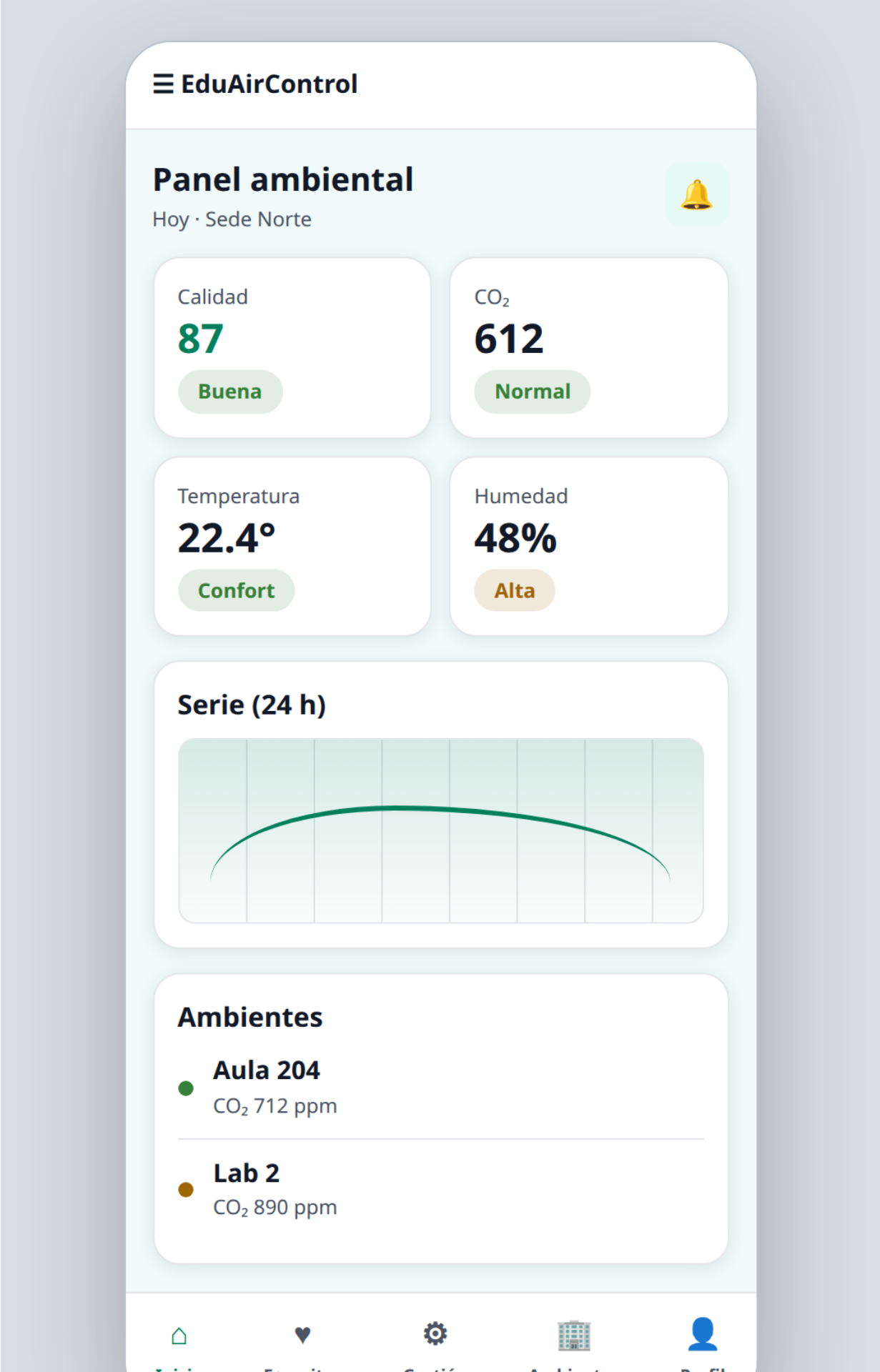
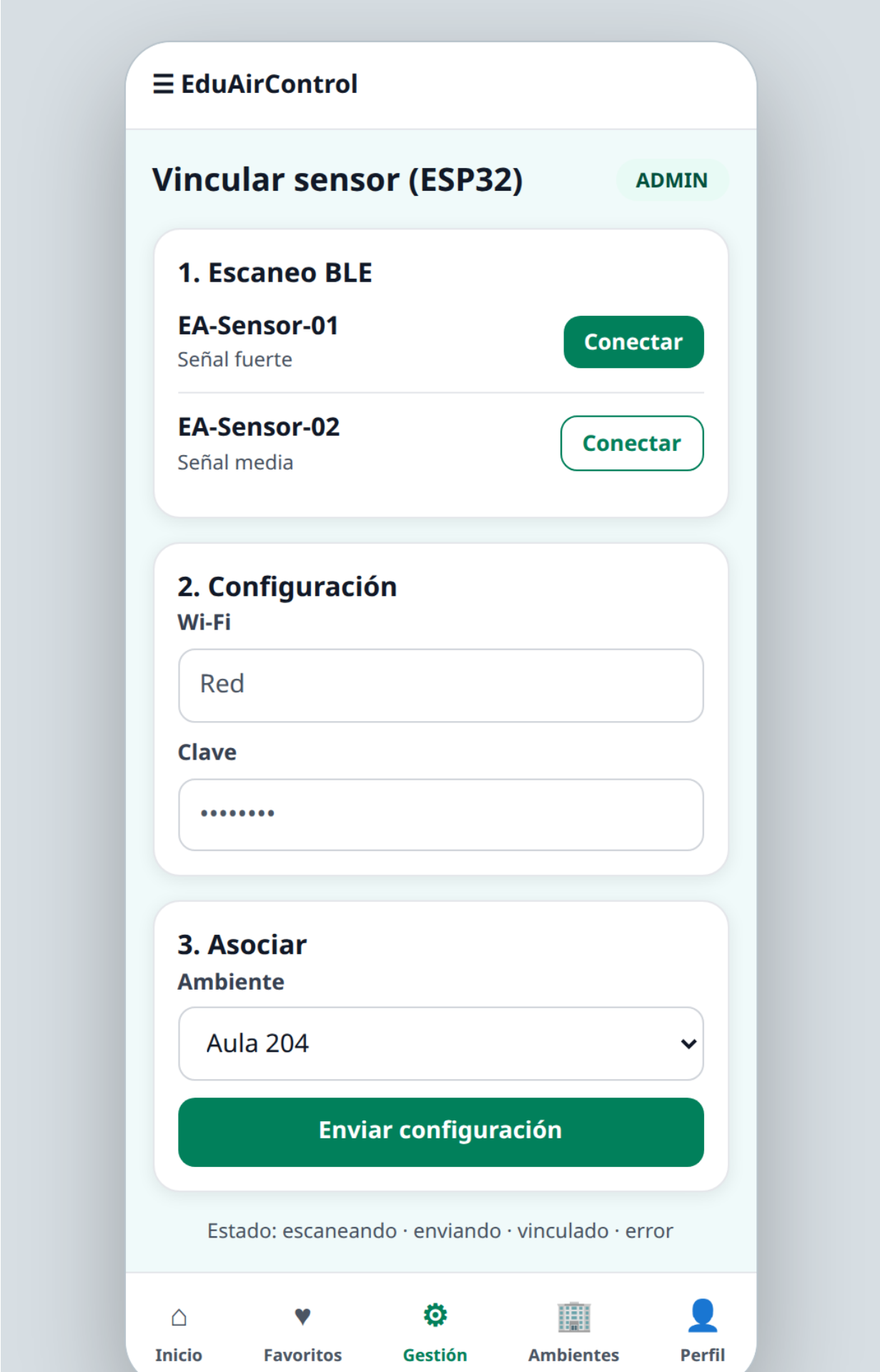

☰ EduAirControl
Panel ambiental
Hoy · Sede Norte
Calidad
87
Buena
CO₂
612
Normal
Temperatura
22.4°
Confort
Humedad
48%
Alta
Serie (24 h)
Ambientes
Aula 204
CO₂ 712 ppm
Lab 2
CO₂ 890 ppm

Figura 9

Provisionamiento de sensor ESP32 por BLE (móvil).


☰ EduAirControl
Vincular sensor (ESP32)
ADMIN
1. Escaneo BLE
EA-Sensor-01
Señal fuerte
Conectar
EA-Sensor-02
Señal media
Conectar
2. Configuración
Wi-Fi
Red
Clave
••••••••
3. Asociar
Ambiente
Aula 204
Enviar configuración
Estado: escaneando · enviando · vinculado · error
Inicio
Favoritos
Gestión
Ambientes
Perfil

2.4 Navegación y control de acceso

La navegación web se define en `App.jsx` con React Router y guardas por rol (`GuestRoute`, `ProtectedRoute`, `AdminRoute`, `SuperAdminRoute`); la móvil se define en `App.js` y `AppNavigator.jsx` con navegación por pestañas y stacks anidados. Los roles implementados en el código son `USER`, `ADMIN` y `SUPER_ADMIN`; el rol `VIEWER` documentado en la especificación no está implementado en el frontend.

Tabla 4

Matriz de acceso por rol

Pantalla	**Invitado**	**USER**	**ADMIN**	**SUPER_ADMIN**
Aterrizaje, guía, términos	✅	✅	✅	✅
Autenticación y recuperación	✅	redirige a `/dashboard`	redirige	redirige
Dashboard, ambientes, favoritos, notificaciones	❌	✅	✅	✅
Perfil y configuración	❌	✅	✅	✅
Administración de ambientes	❌	❌	✅	✅
Panel super administrador	❌	❌	❌	✅
Provisionamiento BLE (móvil)	❌	❌	✅	✅

Las rutas con alias conservan compatibilidad: `/` redirige a `/landing`, `/signup` a `/login?panel=register`, `/ranking` a `/dashboard` y `/admin/users` a `/admin/institutions`.

3. Modelo arquitectónico

3.1 Estilo arquitectónico adoptado

EduAirControl adopta un estilo de **microservicios con API Gateway y comunicación orientada a eventos**, implementado como una migración *strangler fig* desde un monolito heredado. Cada microservicio corresponde a un contexto acotado, posee su propia base de datos lógica y se comunica con los demás mediante REST (sincrónico) o RabbitMQ (asincrónico).

Tabla 5

Características del estilo arquitectónico

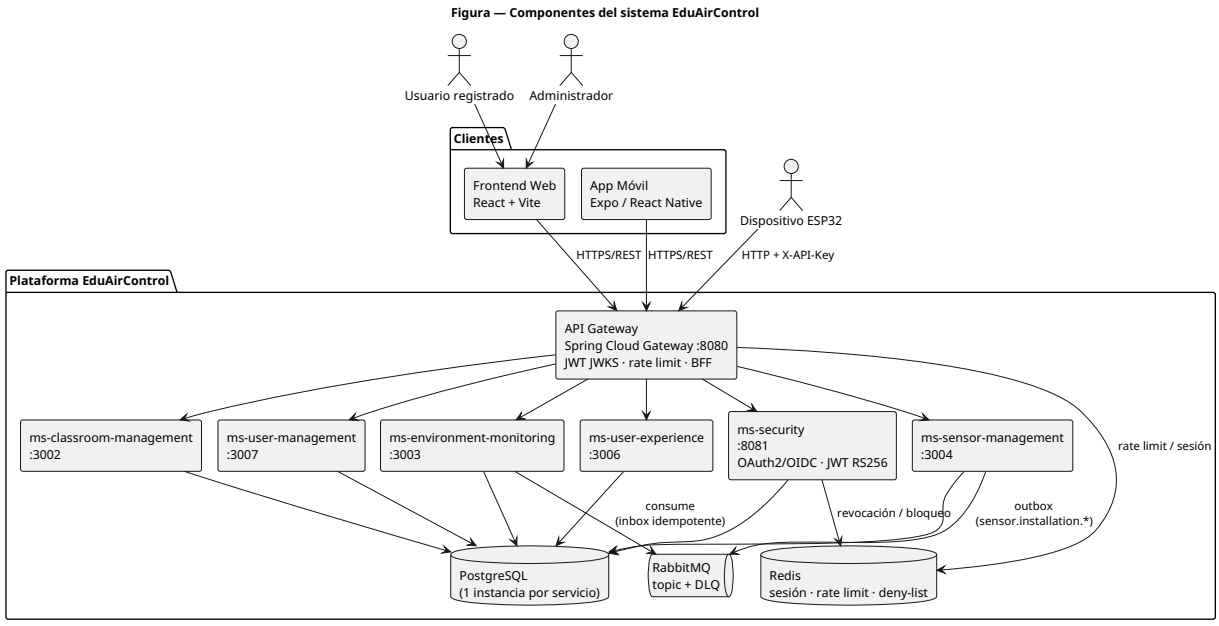
Aspecto	Decisión	Referencia
Descomposición	Un microservicio por contexto acotado (6 servicios)	ADR-002
Entrada única	API Gateway (Spring Cloud Gateway) con validación JWT y rate limiting	ADR-005
Comunicación sincrónica	REST/HTTP con contratos OpenAPI	ADR-004
Comunicación asincrónica	RabbitMQ: intercambio *topic*, outbox transaccional, consumidores idempotentes	ADR-004, ADR-007
Persistencia	PostgreSQL, una instancia física por servicio en el stack `infra`	ADR-003
Arquitectura interna	Hexagonal (puertos y adaptadores) en todos los servicios	ADR-009
Seguridad	OAuth2/OIDC + JWT RS256 con JWKS publicado por `ms-security`	ADR-006
Contenedores	Docker Compose para el stack completo	ADR-010
Observabilidad	Correlation ID en el gateway; stack documentado	ADR-012

3.2 Vista de contexto y de contenedores

En la vista de contexto, los actores son el usuario registrado, el administrador y los dispositivos ESP32. Los clientes web y móviles acceden exclusivamente a través del API Gateway; los dispositivos IoT envían mediciones por HTTP con clave por dispositivo (`X-API-Key`).

Figura 10

*Componentes del sistema EduAirControl.*Clientes, API Gateway, los seis microservicios y los componentes de infraestructura (PostgreSQL, RabbitMQ, Redis).**



3.3 Catálogo de servicios

Tabla 6

Catálogo de servicios, puertos y responsabilidades

Servicio	Puerto	Esquema	Responsabilidad
`api-gateway`	8080	Redis (sesión/ rate limit)	Enrutamiento, validación JWT (JWKS), rate limiting, BFF con cookie `SESSION` httpOnly
`ms-security`	8081	`seguridad`	Registro, autenticación, servidor OAuth2/OIDC, emisión de JWT RS256, rotación de refresh tokens, revocación, instituciones (tenant)
`ms-classroom-management`	3002	`classrooms`	Sedes, tipos de ambiente y ambientes educativos

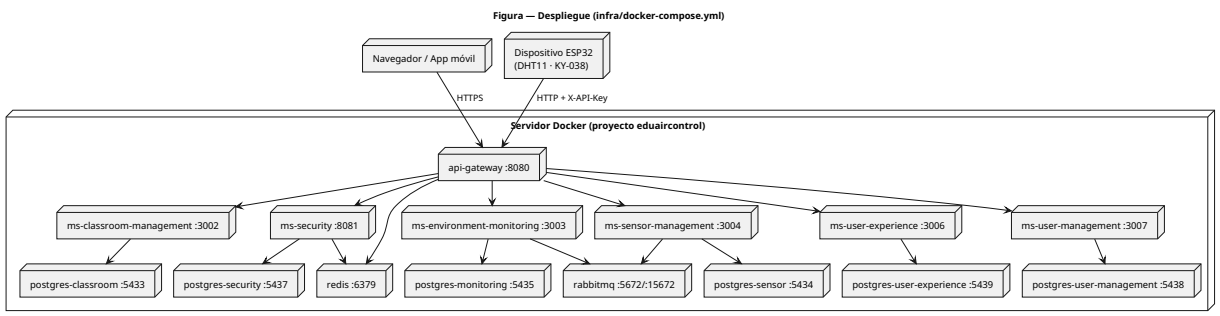
Servicio	Puerto	Esquema	Responsabilidad
`ms-sensor-management`	3004	`sensors`	Sensores, modelos, estados, variables e instalaciones; publica eventos por outbox
`ms-environment-monitoring`	3003	`environment_monitoring`	Mediciones, umbrales, análisis por período, alertas y modelo de lectura del dashboard
`ms-user-management`	3007	`user_management`	Perfiles de usuario
`ms-user-experience`	3006	`user_experience`	Preferencias, favoritos, valoraciones, búsquedas y estado de notificaciones

3.4 Comunicación entre servicios

La comunicación sincrónica usa REST sobre HTTP a través del gateway, que propaga la identidad del usuario mediante cabeceras internas (`X-User-Id`, `X-User-Role`, `X-Institution-Id`, `X-Correlation-Id`). La comunicación asincrónica usa RabbitMQ con dos intercambios *topic*: `eduaircontrol.environmental-data` (mediciones) y `eduaircontrol.sensor` (instalaciones). El patrón de publicación es el outbox transaccional y el de consumo es el inbox idempotente (`ON CONFLICT DO NOTHING`), lo que garantiza al-menos-una-vez sin duplicados efectivos.

Figura 11

*Diagrama de despliegue del stack `infra/docker-compose.yml`.*Una instancia de PostgreSQL por servicio, RabbitMQ y Redis compartidos.**



3.5 Seguridad

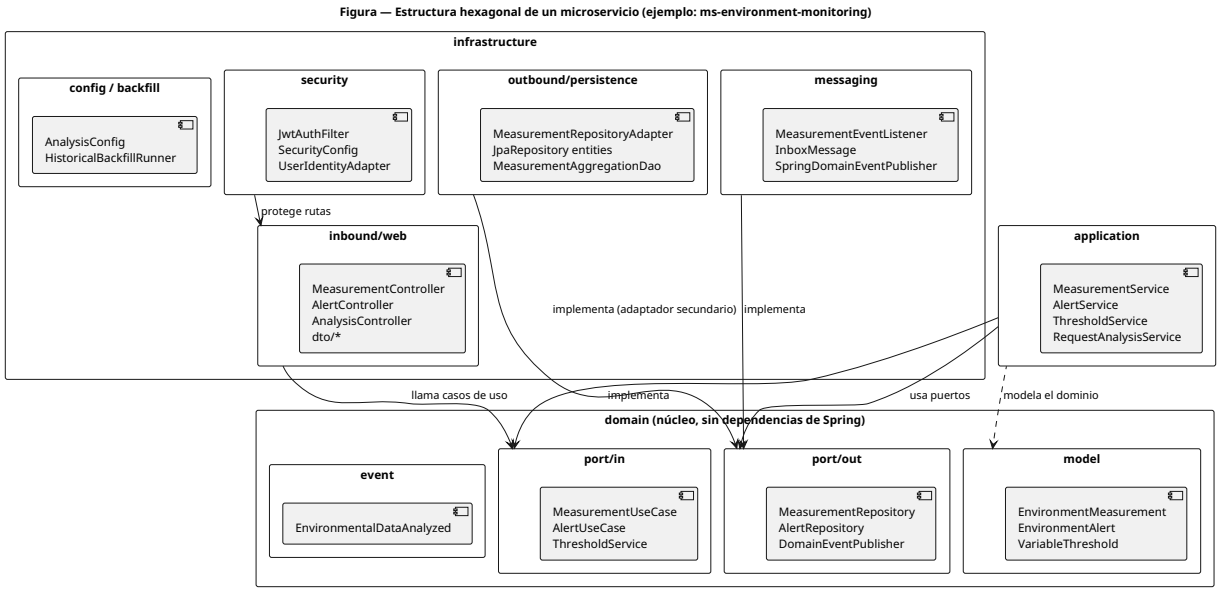
La seguridad se diseña en capas: el gateway valida firma, expiración y lista negra de tokens (almacenada en Redis) antes de enrutar; los servicios aplican autorización fina por rol. Los tokens de acceso son JWT **RS256** firmados por `ms-security`, cuya clave pública se distribuye mediante **JWKS**; los dispositivos IoT usan `X-API-Key` por dispositivo. El BFF del gateway guarda los tokens de la sesión web en Redis y entrega al navegador únicamente una cookie `SESSION` httpOnly, de modo que el token nunca llega a JavaScript.

3.6 Arquitectura interna de los servicios

Cada servicio sigue la arquitectura hexagonal (puertos y adaptadores, ADR-009): el dominio define los modelos y los puertos (`port/in` para casos de uso, `port/out` para repositorios y publicación de eventos), la capa de aplicación implementa los casos de uso y la infraestructura aporta los adaptadores (controladores REST, persistencia JPA, mensajería y seguridad). El dominio no depende de Spring ni de JPA.

Figura 12

Estructura hexagonal de `ms-environment-monitoring`.



4. Patrones de diseño

4.1 Patrones arquitectónicos y de microservicios

Tabla 7

Patrones adoptados y su materialización en el código

Patrón	Propósito	Implementación en el código
Hexagonal (puertos y adaptadores)	Independizar el dominio de la infraestructura	Paquetes `domain/port/in`, `domain/port/out`, `infrastructure/{inbound,outbound}` en los 6 servicios
API Gateway	Entrada única, cross-cutting de borde	`api-gateway` (Spring Cloud Gateway): rutas, JWT, rate limit
BFF (Backend for Frontend)	Sesión de servidor para el navegador	Gateway como cliente OAuth2: tokens en Redis, cookie `SESSION` httpOnly (ADR-017)
Strangler Fig	Migración incremental desde el monolito	Ingesta de mediciones aún en el monolito; instalaciones ya en `ms-sensor-management`
Outbox transaccional	Publicación de eventos fiable junto al cambio de estado	Tablas `outbox_event` en `classrooms` y `sensors`; publicación a RabbitMQ
Inbox idempotente	Consumo al-menos-una-vez sin duplicados	`inbox_message` en `environment_monitoring` con `ON CONFLICT DO NOTHING`
CQRS (modelo de lectura)	Consultas de dashboard eficientes	`environment_measurement`, `installation_projection`, agregaciones (`MeasurementAggregationDao`)
Database per service	Aislamiento de datos por contexto	Un esquema/instancia por servicio; ningún servicio lee tablas ajenas (ADR-003)
Circuit breaker	Resiliencia ante fallos de dependencias	Documentado en ADR-013; **aún no implementado** (deriva registrada)
Retry con backoff	Recuperación ante fallos transitorios	Cola de trabajo con reintento y DLQ en el consumidor de RabbitMQ

4.2 Patrones GoF presentes en el código

Tabla 8

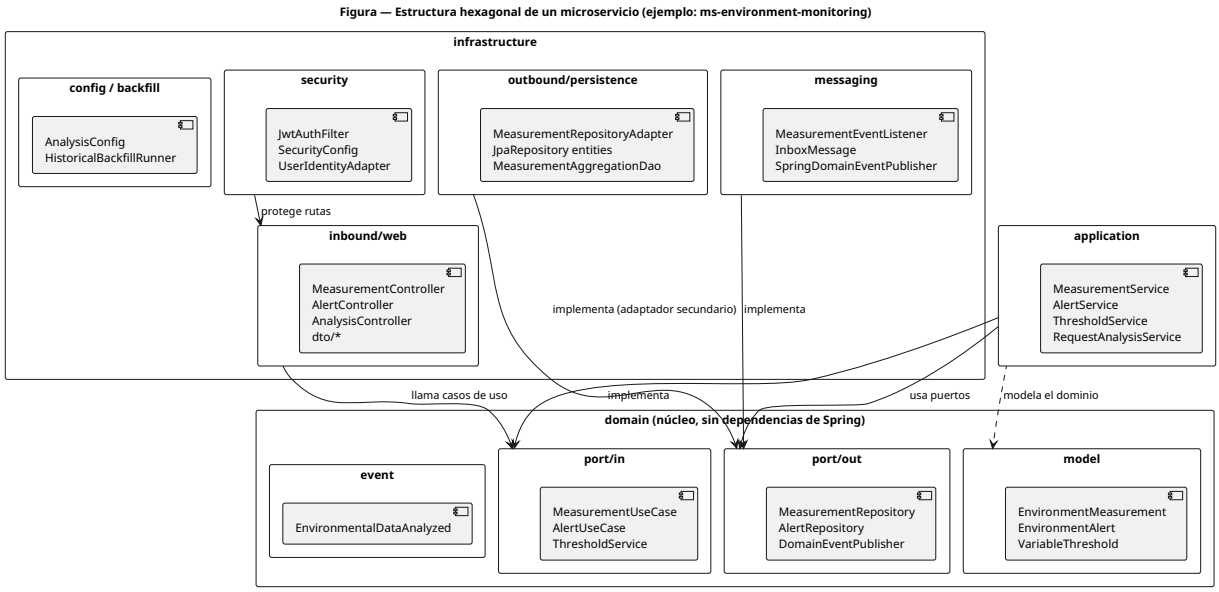
Patrones GoF y su uso concreto

Patrón	Categoría	Uso en EduAirControl
Adapter	Estructural	Adaptadores de persistencia (`*RepositoryAdapter`), `UserIdentityAdapter`, `ThresholdAdapter`
Repository	Estructural/Datos	Interfaces `*Repository` del dominio implementadas con JPA
Builder	Creacional	Entidades de dominio con Lombok `@Builder` (`EnvironmentAlert`, `EnvironmentMeasurement`)
Factory Method	Creacional	Creación de DTOs y de entidades de dominio desde los servicios de aplicación
Observer	Comportamiento	Eventos de dominio (`EnvironmentalDataAnalyzed`) publicados hacia el bus de mensajes
Strategy	Comportamiento	Resolución de períodos de análisis (`PeriodResolver`), evaluación de umbrales
Template Method	Comportamiento	Flujos comunes de los servicios de aplicación (validar → ejecutar → persistir → publicar)
Singleton	Creacional	Beans de configuración de Spring (`AnalysisConfig`, `SecurityConfig`)

4.3 Trazabilidad de patrones en la estructura de paquetes

Figura 13

Trazabilidad del patrón hexagonal en la estructura de paquetes de un servicio.



Cada adaptador primario (controlador) traduce HTTP a un caso de uso; cada adaptador secundario (persistencia, mensajería) implementa un puerto de salida del dominio. Esta separación permite probar el dominio sin infraestructura y sustituir adaptadores (por ejemplo, el publicador de eventos) sin tocar la lógica de negocio.

5. Modelo de datos

5.1 Principios de modelamiento

El modelo de datos se organiza por esquema PostgreSQL, uno por microservicio. Los principios aplicados son: identificadores `uuid` generados por la aplicación, campos de auditoría (`created_at`, `updated_at`, `created_by`, `updated_by`), separación de dominios (ningún servicio lee tablas ajenas), integridad referencial dentro de cada esquema y consistencia eventual entre esquemas mediante eventos.

5.2 Esquemas y tablas principales

Tabla 9

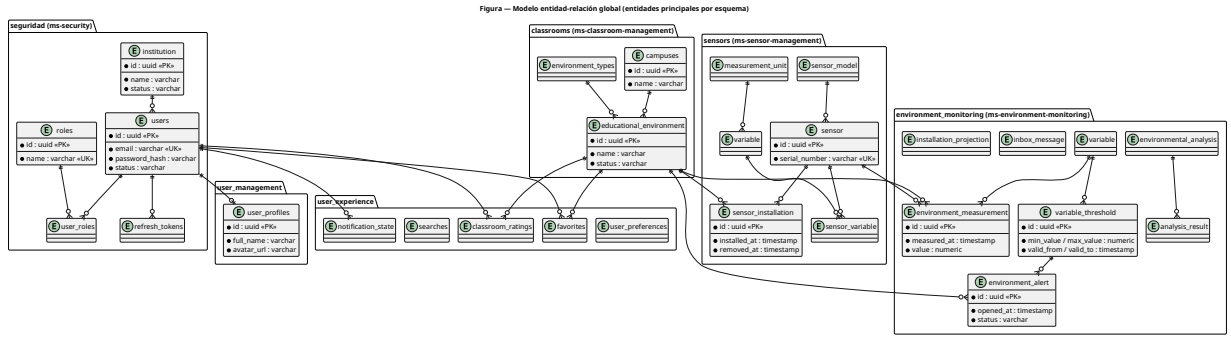
Esquemas y tablas del modelo de datos

Esquema	Servicio	Tablas principales
`seguridad`	ms-security	`institution`, `users`, `user_identity`, `roles`, `user_roles`, `refresh_tokens`, `password_reset_tokens`
`classrooms`	ms-classroom-management	`campuses`, `environment_types`, `educational_environment`, `outbox_event`
`sensors`	ms-sensor-management	`sensor_model`, `sensor_status`, `measurement_unit`, `variable`, `sensor`, `sensor_installation`, `sensor_variable`, `outbox_event`
`environment_monitoring`	ms-environment-monitoring	`variable`, `educational_environment`, `variable_threshold`, `severity`, `quality_flag`, `environment_measurement`, `inbox_message`, `analysis_statuses`, `environmental_analysis`, `analysis_result`, `environment_alert`, `installation_projection`
`user_management`	ms-user-management	`user_profiles`

Esquema	Servicio	Tablas principales
`user_experience`	ms-user-experience	`user_preferences`, `favorites`, `classroom_ratings`, `searches`, `notification_state`

Figura 14

Modelo entidad-relación global (entidades principales por esquema).



5.3 Reglas de negocio del modelo

- **La medición es atómica**: cada fila de `environment_measurement` corresponde a una lectura de una variable, de un sensor, en un ambiente y en un instante.
- **Los umbrales son vigentes en el tiempo**: `variable_threshold` define rangos con `valid_from`/`valid_to`; las alertas referencian el umbral evaluado por clave foránea, lo que preserva la trazabilidad de por qué se generó cada alerta.
- **El análisis es por período**: `environmental_analysis` agrupa resultados (`analysis_result`) por ventana temporal (día, semana, mes, año).
- **Las instalaciones son históricas**: `sensor_installation` registra `installed_at`/ `removed_at`; la eliminación física de un sensor con historial está bloqueada.
- **Los identificadores entre dominios son UUID**: `educationalEnvironmentId`, `sensorId`, `measurementUnitId` se validan como UUID en los servicios frontera, sin validación cruzada de existencia (frontera documentada en `ms-sensor-management`).

El detalle completo del modelo —diccionario de datos y script DDL— se entrega en `06_Base_Datos`.

6. Diagramas de implementación

La estructura de código de todos los servicios sigue el mismo esqueleto hexagonal, lo que permite navegar cualquiera de ellos con las mismas convenciones. El diagrama de componentes (Figura 2) y el de despliegue (Figura 6) completan la vista de implementación; los diagramas UML de casos de uso, secuencia y clases se entregan en `03_Analisis_Software`.

Tabla 10

Convenciones de estructura de código

Capa	Paquete	Regla
Dominio	`domain.model`, `domain.port.{in,out}`, `domain.event`, `domain.service`	Sin dependencias de Spring ni JPA
Aplicación	`application`	Implementa casos de uso; usa solo puertos
Infraestructura	`infrastructure.{inbound.web, outbound.persistence, messaging, config, backfill, security}`	Adaptadores primarios y secundarios
Compartido	`shared.{security, exception, web, contract}`	Seguridad, excepciones y contratos transversales

7. Conclusiones

El diseño de software de EduAirControl consolida cuatro vistas coherentes entre sí y trazables con el código: un sistema de diseño con tokens reales compartidos entre web y móvil y un inventario de doce pantallas; una arquitectura de microservicios con API Gateway, comunicación orientada a eventos y seguridad JWT RS256/JWKS; un conjunto de patrones —hexagonal, repositorio, outbox, inbox, CQRS de lectura— materializados de forma uniforme en los seis servicios; y un modelo de datos por esquema con reglas de negocio explícitas. Las desviaciones conocidas (circuit breaker documentado e no implementado, deriva entre ADR-003 y el despliegue, rol `VIEWER` solo documentado) quedan registradas como entradas del entregable de calidad y mejoras. El diseño sirve como referencia para la implementación continua, el plan de pruebas y la operación del sistema.

Referencias

- EduAirControl. (2026). *Documento general del proyecto* [Entregable 01].
- EduAirControl. (2026). *Especificación de requisitos de software* [Entregable 02].
- EduAirControl. (2026). *Análisis de software* [Entregable 03].
- EduAirControl. (2026). *Propuesta técnica* [Entregable 04].
- EduAirControl. (2026). *Modelo entidad-relación, diccionario de datos y script de base de datos* [Entregable 06].
- EduAirControl. (2026). *Documentación de gobernanza: arquitectura del sistema, guía de patrones y arquitectura hexagonal* (`docs-ea-control/05-architecture`).
- EduAirControl. (2026). *Modelos de datos por dominio* (`docs-ea-control/06-data`).
- EduAirControl. (2026). *Sistema de diseño, mapa de navegación y mockups* (`docs-ea-control/12-ux-ui`).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.